Remedy Mesh - Solution Document

Multi-Agent Architecture on Vertex AI Agent Engine

Executive Summary

Remedy Mesh is a two-layer agentic system that watches closed automotive recalls against live field evidence, proves - with deterministic, auditable logic - when a remedy's scope has quietly expanded, and then runs a real multi-supplier parts-sourcing negotiation to secure replacement capacity fast, with a human approver standing between every agent-drafted artifact and the real world.

Its central architectural strength is genuine multi-agent reasoning wrapped in verified safety guardrails. Eight Gemini 2.5 agents interpret evidence, independently audit each other's conclusions, and negotiate a real multi-supplier deal in real time - while a built-in verification layer keeps every triggering decision and every unit allocation safe, consistent, and fully auditable. This is what makes an autonomous negotiation mesh trustworthy enough to act on in the real world: the agents do the reasoning, and the guardrails make it safe to let them.

This document describes the **Vertex AI Agent Engine architecture**: 8 Google ADK agents, each deployed as an independent managed service on Vertex AI Agent Engine, called from a Cloud Run orchestrator (remedymesh) that reads case data from a 603-record Firestore database. All 9 demo cases, and real outbound Twilio confirmation calls to simulated supplier personas, have been run live against this deployed stack (§12, Verification and Test Results).

The system has two layers:

- **Layer 1, Remedy Watchdog** - an Evidence Interpreter, Remedy Investigator and Verification Agent chain that independently audits every verdict against the case's

lot, complaint and policy records, and flags any disagreement to a person for review.

- **Layer 2, NegotiationMesh** - an isolated, sealed-bid negotiation mesh in which three supplier agents and a ceiling-enforcing OEM counter-agent converge on a unit allocation in real time, confirmed in the real world only through a human-approved outbound phone call.

This document sets out, end to end, what Remedy Mesh is, how it is built, how it runs, and what has been verified - a working answer to a real, open problem across the automotive recall industry: no existing tool continuously re-tests whether a shipped remedy actually worked, and getting replacement capacity committed once it hasn't is still a slow, manual process. Remedy Mesh closes both gaps, and is built so the pattern generalizes well beyond this one use case.

Problem Statement

A recall's remedy shipping is treated, almost everywhere in the industry's tooling, as the end of the story. Opcenter, Teamcenter, SAP Defect Management and Catena-X all record *what happened* - which VINs were notified, which dealers performed the fix - but none of them continuously re-test *whether the remedy actually worked* against the field evidence that keeps arriving after the fix ships. That gap is where recalls quietly re-open, expand, or get re-triggered months or years later, after enough new failures accumulate that a regulator or the manufacturer is forced to act - by which point the population affected, and the cost of fixing it, has grown far beyond the original scope.

This is not a hypothetical pattern. It is the documented behavior of real, recent recalls:

Case	What happened
BMW i4 left rear side beam recall (NHTSA 24V-611)	BMW's own filing states the previously identified containment range needed to be expanded after a supplier production-date check - a population revision discovered after the fact, not predicted.

Case	What happened
BMW Integrated Brake System recall	Expanded in multiple rounds across 2023–2025 - roughly 80,000 → 278,000 → approximately 1.5 million BMW/Mini vehicles - as more field failures surfaced after each prior remedy.
BMW–Aumovio settlement, August 2026	Aumovio (the automotive unit Continental spun off in 2026) paid BMW €350 million settling litigation over the underlying brake defect - a real, material cost attached to a recall whose scope had already expanded repeatedly.
GM L87 engine investigation (NHTSA EA26005, opened August 20, 2026)	997,743 vehicles under investigation after 690 new failure reports - 499 of them on vehicles where the revised remedy had *already* been applied, and 191 on engines outside the original recall's production window. This is exactly the failure mode Remedy Mesh is built to catch earlier: new field evidence showing a shipped remedy did not fully close the defect.

These four data points are independently verified against primary sources - NHTSA recall filings, Baker McKenzie's reporting on the Aumovio settlement, and GM Authority's reporting on NHTSA's own EA26005 filing - rather than carried over from secondary summaries, so every figure cited above is traceable back to its original source.

These are real, independently verified recall patterns used to motivate the problem - not a claim that Remedy Mesh would have changed any of these four specific outcomes, that a language model can determine legal recall scope on its own, or that existing recall, quality and sourcing platforms have no overlapping functionality. The narrower claim here is operational: once a scope-drift signal exists in the incoming evidence, nothing in the current toolchain continuously re-tests for it, and nothing automates the multi-supplier sourcing response that should follow once it is confirmed.

Two compounding failures sit behind every one of these cases. First, **evidence blindness**: nobody is systematically re-scoring incoming complaints, warranty claims and

production-lot records against the original remedy's assumptions, so a drifting failure rate has to become large and undeniable before anyone acts on it. Second, even once scope expansion *is* recognized, **the sourcing response is slow**: getting replacement-part capacity committed across multiple suppliers, under a real cost ceiling, with real confirmation that each supplier can actually deliver, is a manual, sequential, days-to-weeks process - exactly when speed matters most for owner safety and manufacturer liability.

Remedy Mesh addresses both halves of this problem in one pipeline: continuous evidence verification that decides *whether* scope has genuinely expanded, followed immediately - only when that threshold is actually crossed - by an automated, isolated multi-supplier negotiation that gets replacement capacity committed in seconds rather than weeks, with a human approver still required before anything real is sent. This is a pattern any OEM, Tier-1 supplier or regulator could apply the moment a remedy ships - turning an industry-wide blind spot into a problem that gets caught and fixed automatically.

Solution Overview

Remedy Mesh is a case-management application, that runs two distinct AI-driven layers back to back, with a built-in safety gate between them that keeps the expensive negotiation stage reserved for cases that genuinely need it.

Layer 1 - Remedy Watchdog (evidence verification). For any open or re-opened case, three chained agents - an Evidence Interpreter, a Remedy Investigator and a Verification Agent - read the case's complaint, lot, campaign and policy records and build an evidence picture: deduplicated complaint counts, a post-remedy failure ratio against a policy threshold, geographic clustering, and whether the paperwork trail (install dates, VIN and lot matches) is complete. A deterministic function, evaluate_case(), turns that evidence into exactly one of seven verdicts - confirmed_trigger_mesh, chase_record, remedy_quality_flag, monitor, keep_open, insufficient_evidence, or no_signal - and the Verification Agent independently audits that same verdict against the records, flagging any disagreement for a person rather than silently resolving it.

The gate. Only a confirmed_trigger_mesh verdict proceeds to Layer 2. Every other verdict ends in a human-facing recommendation - chase a missing record, flag for engineering review, keep monitoring, request more evidence, or dismiss - and stops there. This gate is the system's single most important design decision: the expensive, dramatic negotiation mesh never runs unless the deterministic evidence genuinely crosses the threshold.

Layer 2 - NegotiationMesh (supplier negotiation mesh). Once triggered, a Supplier Offer Agent runs once per supplier, each in its own isolated session with zero visibility into any other supplier's session or offer. An OEM Counter Agent, given only the OEM's real deterministic ceiling price and a count (never the content) of alternative offers, counters any offer above that ceiling. Deterministic code then allocates the confirmed unit requirement across the suppliers who converged at or under the ceiling. The allocation is confirmed in the real world through an outbound Twilio phone call per committed supplier, read by a dedicated Call Understanding Agent; a decline triggers a live re-source and re-negotiation against the reserve supplier, never a silent acceptance of its opening price.

Maker-checker governance throughout. An Analyst (Eleanor) runs cases live and places confirmation calls; an Approver (James) only ever replays the stored result of a run - never a re-rolled version - and is the only role who can approve a Part 573 regulatory amendment, send a purchase order, or acknowledge a decline. No drafted artifact reaches the outside world without that second, separate person's sign-off.

The following sections work through why this needs multiple cooperating agents rather than one model call or rules alone (§4), the cloud architecture that runs it (§5), each of the 8 agents in the running application (§6), how the negotiation mesh actually works (§7), a full case trace (§8), the governance model (§9), reliability engineering (§10), cost (§11), what has actually been verified (§12), where the product is headed next (§13), and, to close, the real-world impact of putting this into production (§14).

Why Multi-Agent Generative AI Is Required

It is fair to ask why this problem needs eight separate Gemini agents rather than one well-prompted LLM call, or no LLM at all. Two different answers apply to the two layers.

Why not rules alone? Rules alone are exactly what evaluate_case() already is, and they are deliberately kept in sole charge of the decision - that is the system's core integrity guarantee. What rules alone cannot do is read unstructured, inconsistent, free-text evidence (a complaint description, a dealer's lot-chase note, a supplier's spoken reply on a phone call) and turn it into the structured claims the deterministic function needs, or independently sanity-check that function's own verdict against the same messy evidence from a different angle. That is exactly the gap each agent fills, and no more.

Why not one LLM call? A single large prompt asked to "read the evidence, decide, and negotiate" would collapse three properties this system depends on:

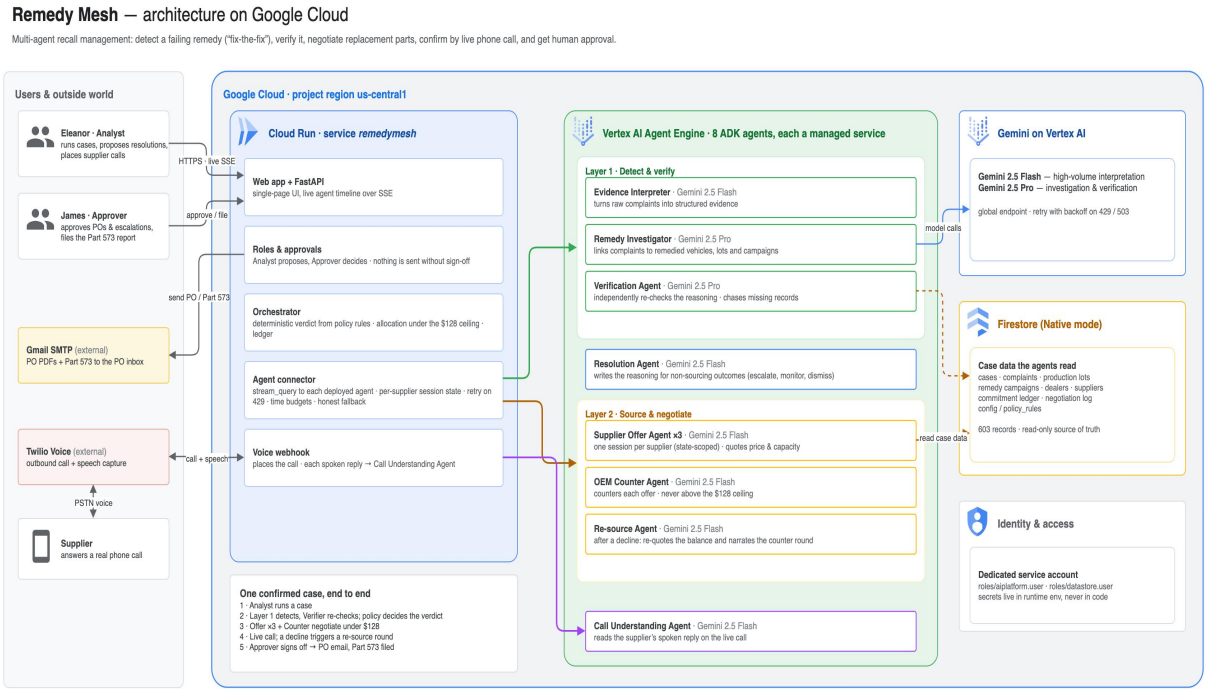
- **Independent audit.** The Verification Agent's value is specifically that it re-derives a verdict from the same records *without* seeing the deterministic function's reasoning, so a disagreement is a genuine second opinion, not the same reasoning restated. A single call cannot audit itself.
- **Supplier isolation.** The entire negotiation mesh's credibility rests on each supplier persona having zero visibility into any other supplier's session, cost floor, or offer - exactly as a real OEM's confidential, parallel sourcing conversations work, specifically so suppliers cannot collude or anchor off each other. A single shared context collapses that isolation guarantee by construction: if one model instance sees all three suppliers' numbers, there is no actual isolation to point to, whatever the prompt claims.
- **Resilience by design.** Running each stage as a small, independently retryable call means any one agent can be retried without ever losing the rest of the run. Eight small, independently retryable calls scale gracefully under real cloud conditions; one large call would be all-or-nothing.

What each agent specifically contributes:

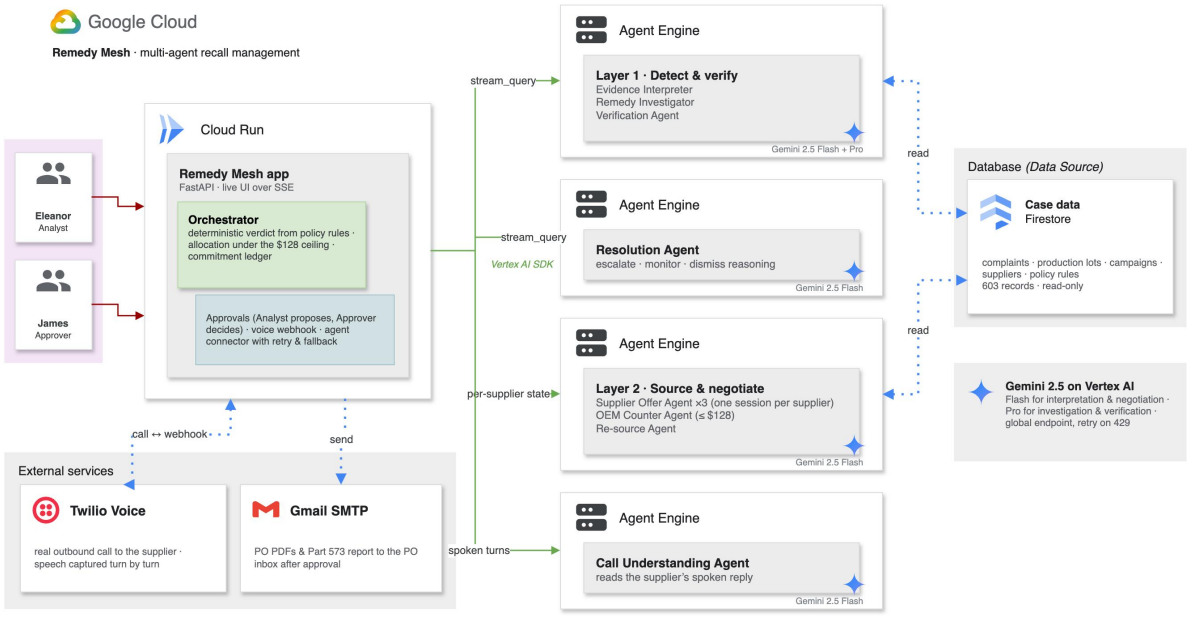
Agent	Contributes	A rules engine or single prompt cannot, because
Evidence Interpreter	Turns raw complaint/lot text into structured claims	Free-text interpretation is exactly what LLMs are for and rules are not
Remedy Investigator	Chases missing lot/production records and reasons about what's missing	Requires judgment about incomplete evidence, not a fixed rule
Verification Agent	Independently re-derives the verdict and flags disagreement	Must not share the deciding function's own reasoning path
Resolution Agent	Narrates the verdict in plain language for a human reader	Deterministic output is a verdict label, not a readable explanation
Supplier Offer Agent	Negotiates from one supplier's private cost floor and capacity	Must have zero visibility into any other supplier's session
OEM Counter Agent	Counters toward the ceiling without ever exceeding it	Needs to react to each supplier's specific offer language, within a hard limit
Call Understanding Agent	Reads a real spoken reply into a structured outcome	Natural speech, not a fixed grammar
Re-source Narration Agent	Explains a live re-negotiation after a decline	Needs to narrate a dynamic, just-computed outcome in context

The thesis that ties all eight together is the same one stated in the Executive Summary: genuine multi-agent reasoning, kept safe and auditable by a built-in verification layer that makes sure the system's decisions and allocations stay trustworthy, every time.

Architecture on Google Cloud



Remedy Mesh - detailed architecture on Google Cloud, showing Cloud Run, Vertex AI Agent Engine's 8 agents, Firestore, Gemini on Vertex AI and IAM



Remedy Mesh - simplified architecture overview

This architecture's headline change from the original, in-process build is *where the agents run*: instead of all agent logic executing as library calls inside one Cloud Run container, each of the 8 agents is deployed as its own independent managed resource on Vertex AI Agent Engine, called remotely from the orchestrator.

Component	Role
Cloud Run service remedymesh	The FastAPI orchestrator: frontend, REST + live SSE feed, maker-checker sign-in, the deterministic verdict gate (evaluate_case), the negotiation core (allocation, ledger, re-source), the agent connector, the Twilio voice webhook, PO and Part 573 PDF drafting, and the approval queue - all in one container, with its own dedicated service account (roles/aiplatform.user + roles/datastore.user only)
Vertex AI Agent Engine	8 Google ADK LlmAgents, each deployed as agent_engines.create(AdkApp(agent=…, enable_tracing=True)) from the real, unmodified agent code - not a rewrite - updated in place after any code change so resource identities are stable
Gemini 2.5 on Vertex AI	Flash for most agents, Pro for the Remedy Investigator and Verification Agent; the **global** Vertex endpoint is used specifically for availability under shared capacity pressure (§10)
Firestore (Native mode, us-central1)	The read-only case database: 9 cases, 503 complaints, 30 production lots, 7 recall campaigns, 4 parties, 50 dealers and config/policy_rules - 603 records in all, read by the app and by the 4 data-reading agents
Twilio Voice	Real outbound confirmation calls to the supplier personas; spoken replies return to a webhook read by the Call Understanding Agent
Gmail SMTP	Sends purchase orders and files the Part 573 amendment - only after the Approver has signed off

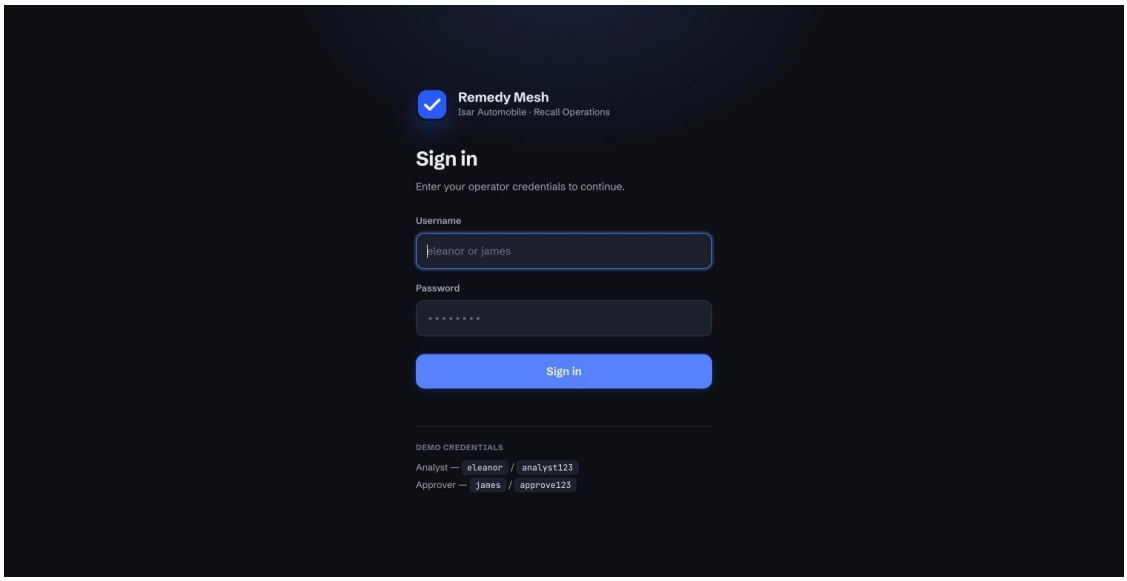
All 9 case verdicts were verified to match the original JSON-backed version exactly after the Firestore migration (§12).

Every agent turn opens a fresh, short-lived session; deliberately, **no cross-run Memory Bank is used** - persistent memory across runs would weaken the supplier isolation guarantee that the negotiation mesh's credibility depends on (§7). This is a considered design choice: supplier-reliability memory across recalls is on the roadmap (§13), as a way to make the mesh even smarter over time.

The 8 Agents - Full Specification

All screenshots below were captured live against the deployed remedymesh service in this session, by driving the real browser UI - none are mockups.

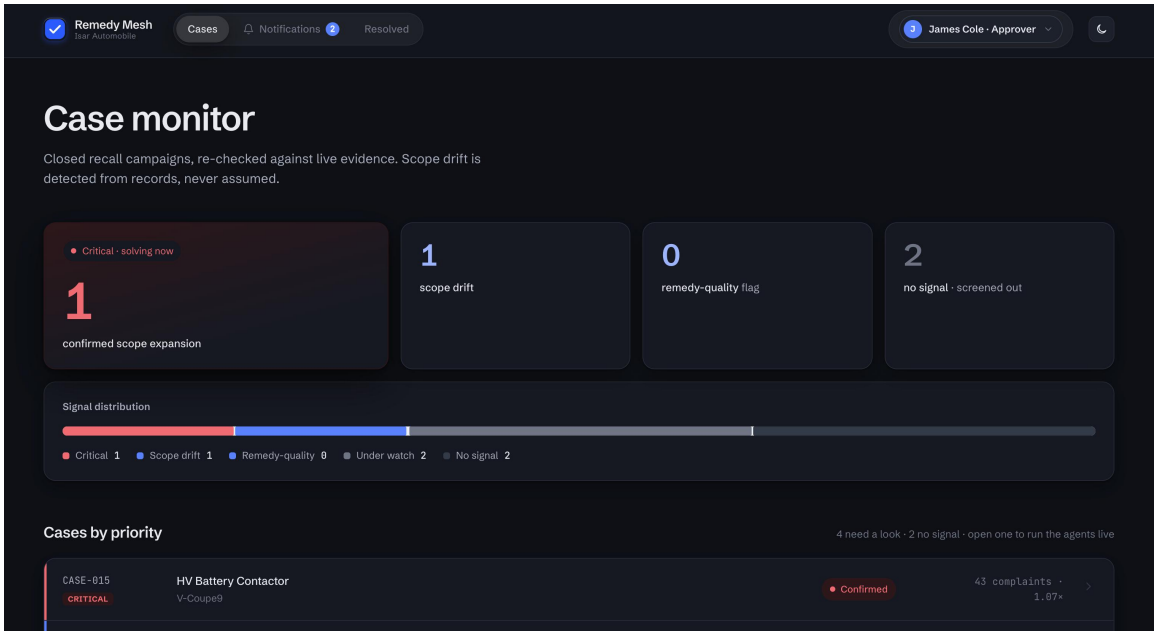
6.1 Sign-in and role separation



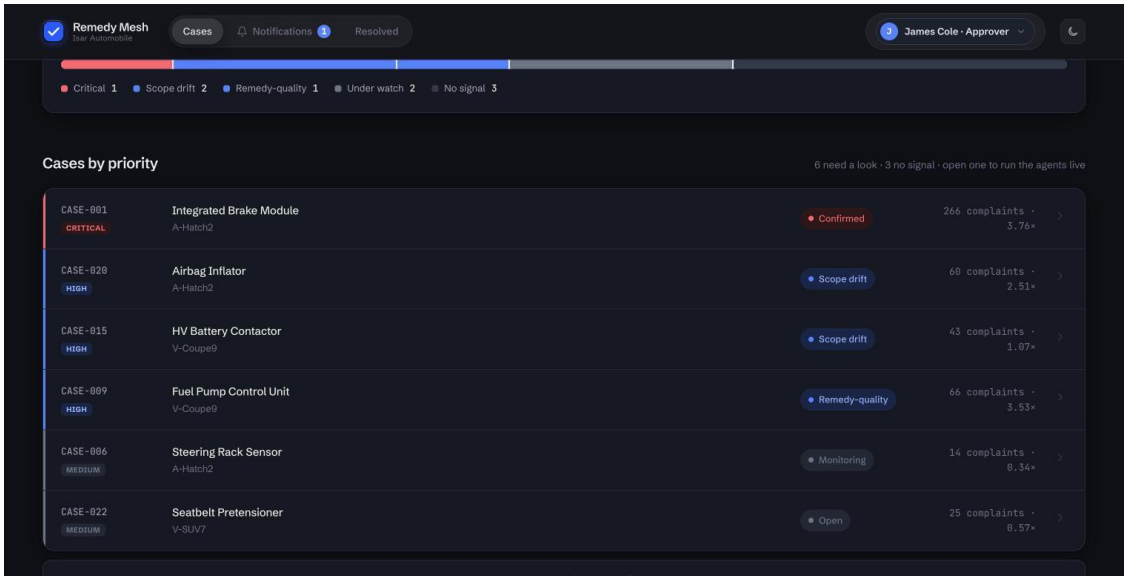
Remedy Mesh sign-in screen, Isar Automobile Recall Operations, showing demo credentials for both roles (Sign in to our app using the credentials in the image)

The application is framed as an internal tool for "Isar Automobile, Recall Operations." Two demo identities exist from the first screen a user sees: Eleanor Ashworth (Analyst) and James Cole (Approver). This is not a cosmetic login - every later screen enforces which actions each role can take, down to disabling specific buttons server-side, not just hiding them in the UI (§9).

6.2 Case Monitor dashboard



Case Monitor dashboard top stats: 1 confirmed scope expansion, 2 scope drift cases under watch

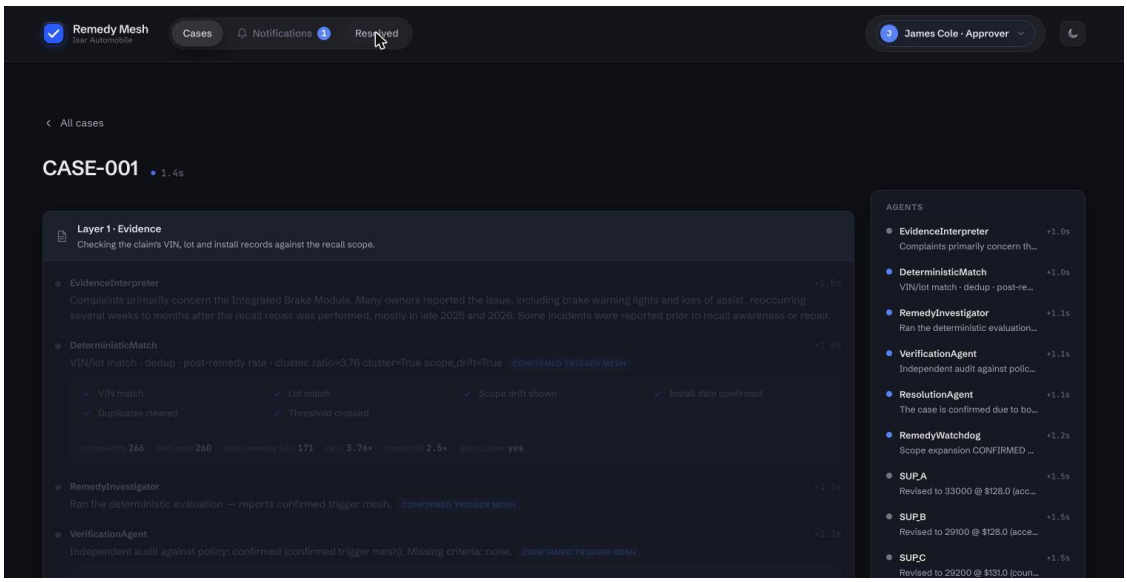


Full case list by priority, with a signal-distribution bar across Critical, Scope drift, Remedy-quality, Under watch and No-signal cases

The dashboard is Eleanor's working view across all open cases, ranked by priority and tagged with the Layer-1 signal each has produced so far: a confirmed scope expansion

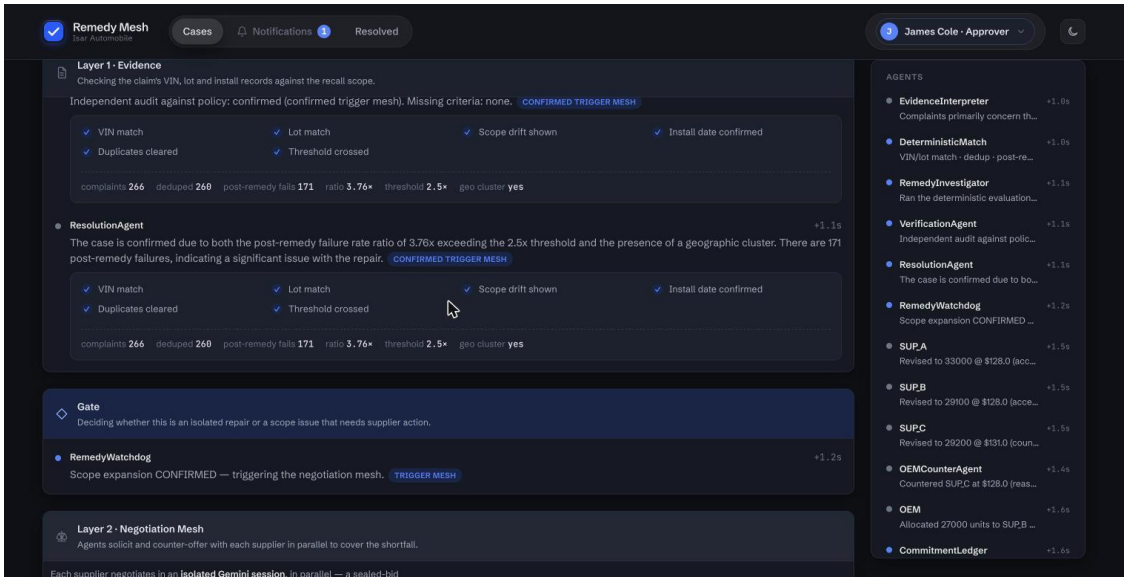
(CASE-001, 266 complaints, a 3.76× post-remedy failure ratio against threshold), cases still chasing records (CASE-020, CASE-015), a remedy-quality flag (CASE-009), monitoring cases (CASE-006), and cases with no signal. This view exists so an Analyst never has to open every case to know where attention is needed - the deterministic verdict has already done the triage.

6.3 Layer 1 - the evidence chain (Remedy Watchdog)



CASE-001 Layer 1 evidence header: Evidence Interpreter and DeterministicMatch showing a CONFIRMED TRIGGER MESH badge with the full criteria checklist

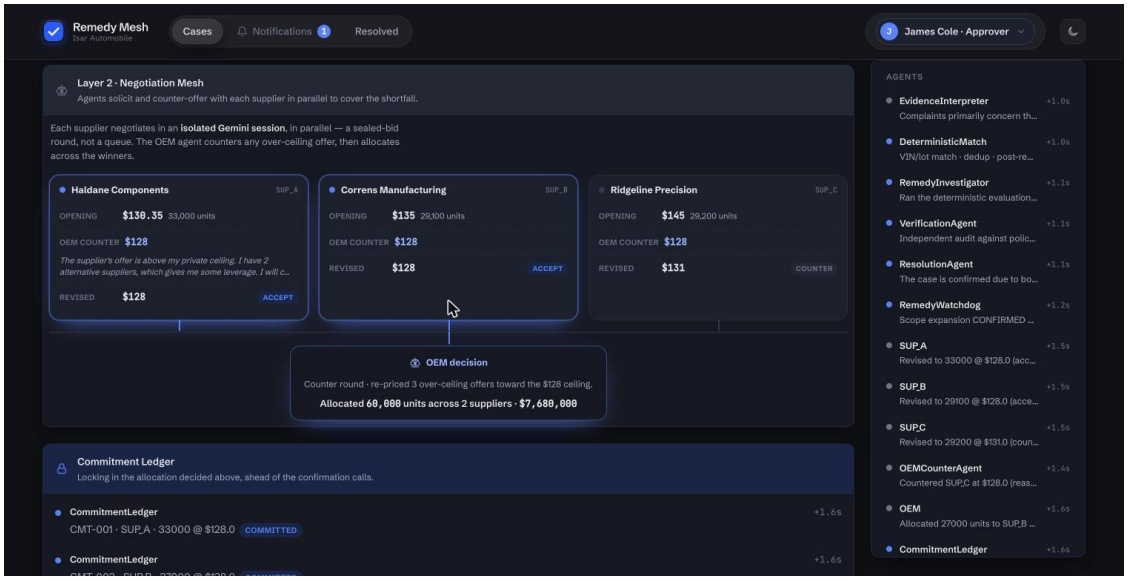
This is the start of a live Layer-1 run on CASE-001 (Integrated Brake Module, A-Hatch2). The Evidence Interpreter's structured read of the complaint and lot records feeds the deterministic match, which checks six criteria in the open: VIN match, lot match, scope drift shown, install date confirmed, duplicates cleared, and threshold crossed. The numbers behind the badge are shown, not hidden - 266 raw complaints, 260 after deduplication, 171 post-remedy failures, a 3.76× ratio against a 2.5× policy threshold, and a confirmed geographic cluster.



The full Layer 1 chain: Remedy Investigator, Verification Agent's independent audit, Resolution Agent's verdict narration, the Gate confirming TRIGGER MESH, and the start of the Layer 2 negotiation mesh

The rest of the chain runs in view: the Remedy Investigator reasons over the chased lot records, the Verification Agent performs its **independent** audit and reports "missing criteria: none" - agreeing with, not restating, the deterministic verdict - and the Resolution Agent narrates the verdict in plain language. The Gate then shows its own explicit reasoning: "Scope expansion CONFIRMED - triggering the negotiation mesh," tagged **TRIGGER MESH**, immediately above the Layer 2 section's own framing: "Each supplier negotiates in an isolated Gemini session, in parallel - a sealed-bid round, not a queue."

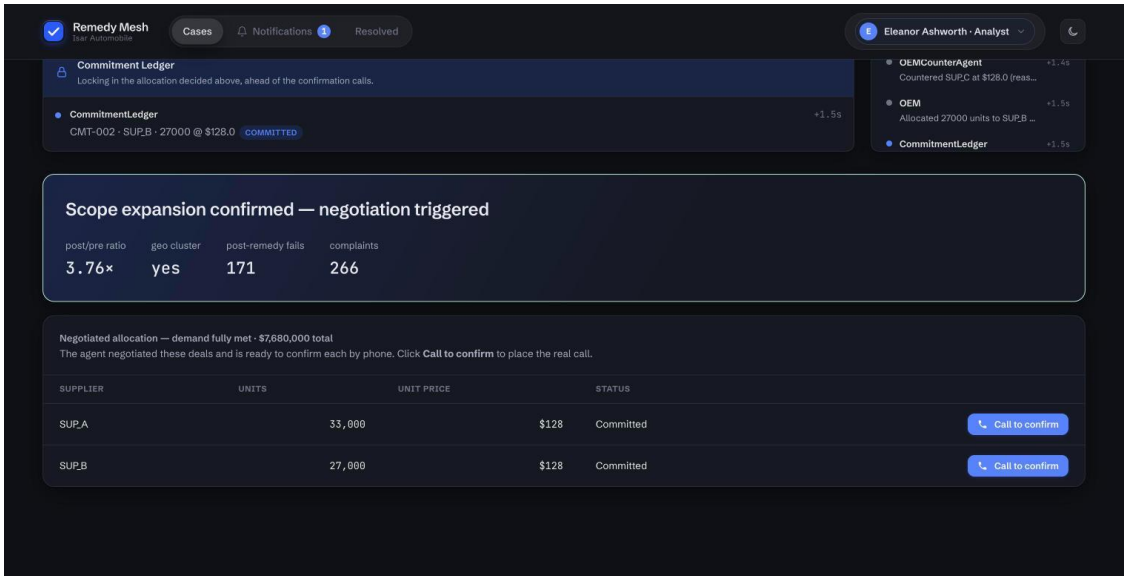
6.4 Layer 2 - the negotiation mesh (CommitMesh)



The three supplier lane cards for CASE-001 converging under OEM counters toward the $128 ceiling, ending in an OEM allocation decision

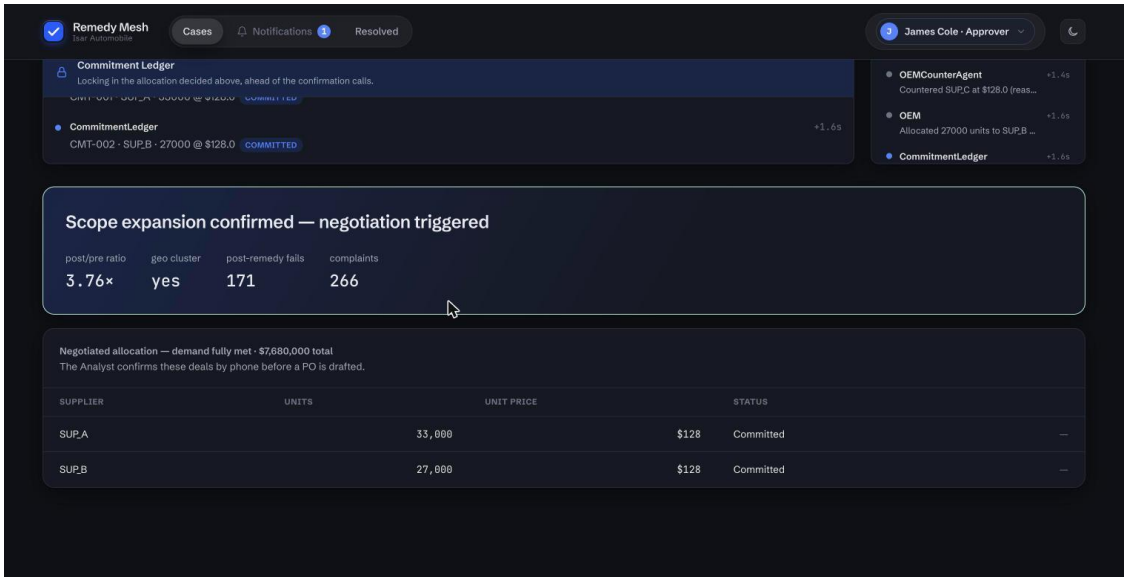
This is the system's signature visual, and it is worth reading carefully because it is the clearest evidence of how the negotiation mesh behaves under real supplier constraints (§7 covers that directly). Three supplier lanes run in parallel, each the output of one isolated Supplier Offer Agent session: Haldane Components opens at $130.35 for 33,000 units, is countered to the $128 ceiling, and revises to accept at $128; Correns Manufacturing opens at $135 for 29,100 units, is countered to $128, and accepts at $128; Ridgeline Precision opens at $145 for 29,200 units, is countered to $128, but only revises to $131 - still over ceiling - and is held as an unallocated reserve rather than awarded. The OEM decision box beneath summarizes what the deterministic allocator actually did: "Counter round · re-priced 3 over-ceiling offers toward the $128 ceiling. Allocated 60,000 units across 2 suppliers · $7,680,000." Every number in that sentence is arithmetic, not a model's claim about itself.

6.5 Allocation table and the maker-checker boundary



The allocation table as Eleanor (Analyst), with real, enabled "Call to confirm" buttons for both committed suppliers

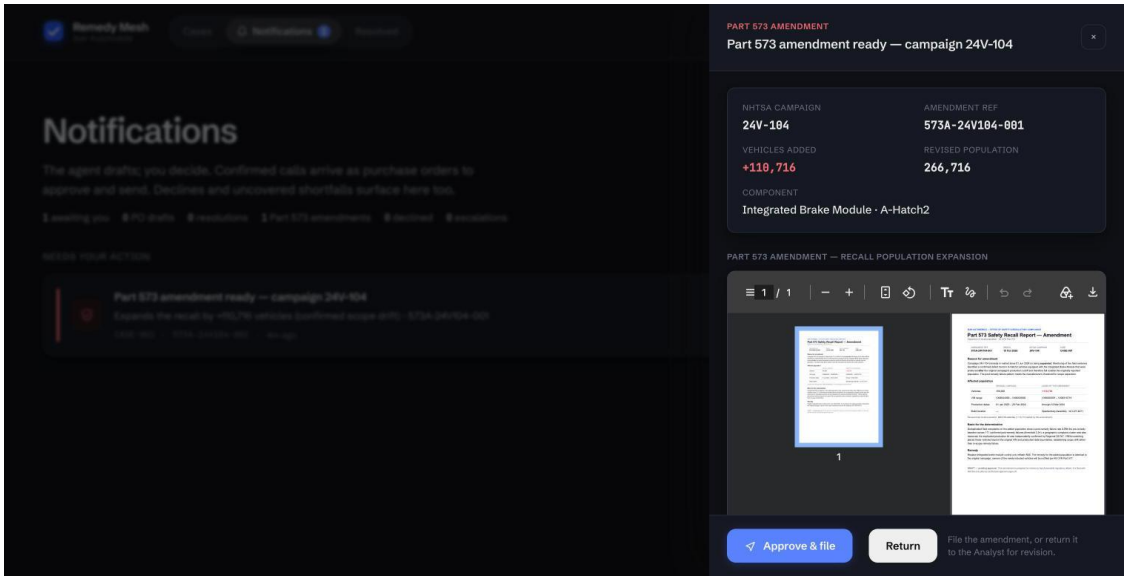
As Eleanor, the allocation table carries real, enabled **Call to confirm** buttons for both committed suppliers - Haldane (33,000 @ $128, marked Committed) and Correns (27,000 @ $128, marked Committed) - with explicit copy: "The agent negotiated these deals and is ready to confirm each by phone. Click Call to confirm to place the real call."



The same allocation table as James (Approver): call buttons replaced with dashes, with copy explaining only the Analyst can place that call

The same table, viewed as James, shows the identical data with the call buttons replaced by dashes and the copy: “The Analyst confirms these deals by phone before a PO is drafted.” This is the clearest proof in the whole application that role separation is enforced at the server, not just styled differently in the frontend - the same case, the same allocation, a structurally different set of available actions per role.

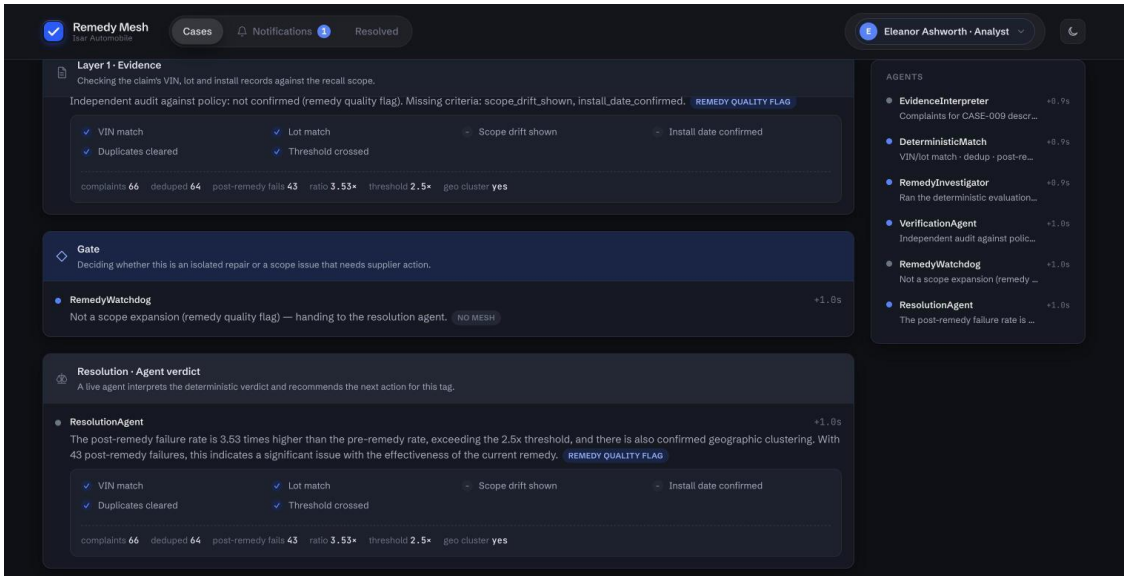
6.6 Part 573 regulatory amendment review



The Part 573 amendment review drawer as James, showing the real NHTSA campaign reference, the population increase, and an enabled "Approve & file" button

This drawer is where a scope-expansion case becomes a real regulatory artifact: NHTSA Campaign 24V-104, Amendment Reference 573A-24V104-001, a population increase of +110,716 vehicles to a revised total of 266,716, naming the affected component (Integrated Brake Module · A-Hatch2), with a rendered two-page amendment PDF preview. James's enabled **Approve & file** button, alongside a **Return** option to send the case back to the Analyst for revision, was likewise left unclicked during capture. The equivalent view as Eleanor shows this same button disabled and greyed, captioned "Only an Approver can act on this" - the same boundary enforced again, on the single highest-stakes action in the system.

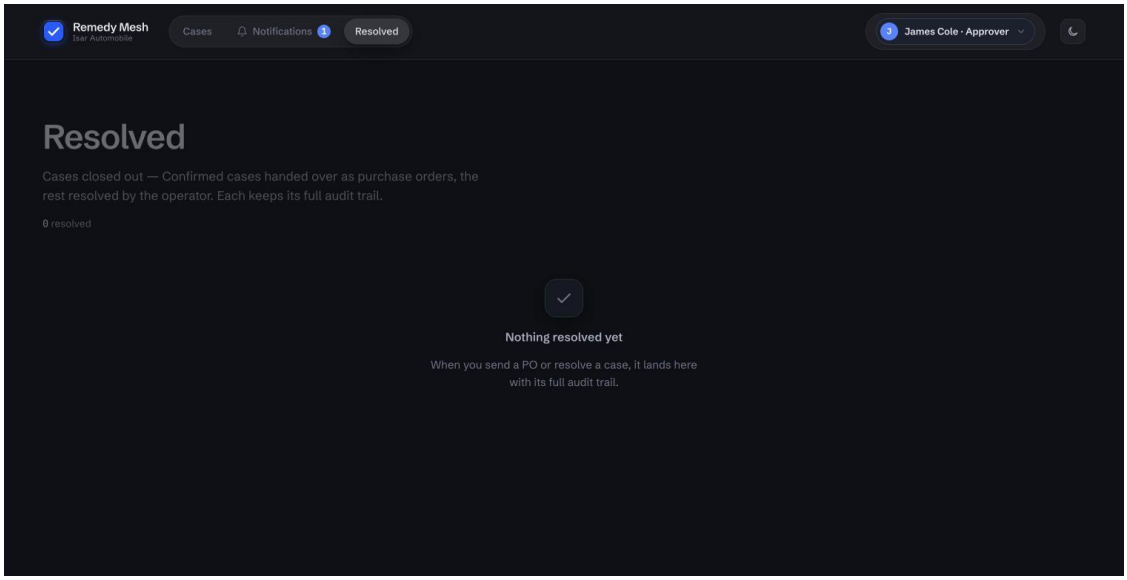
6.7 A contrasting case - correctly not triggering the mesh



CASE-009 (Fuel Pump Control Unit): the full Layer 1 chain correctly tagging a REMEDY QUALITY FLAG and routing to NO MESH, not a scope expansion

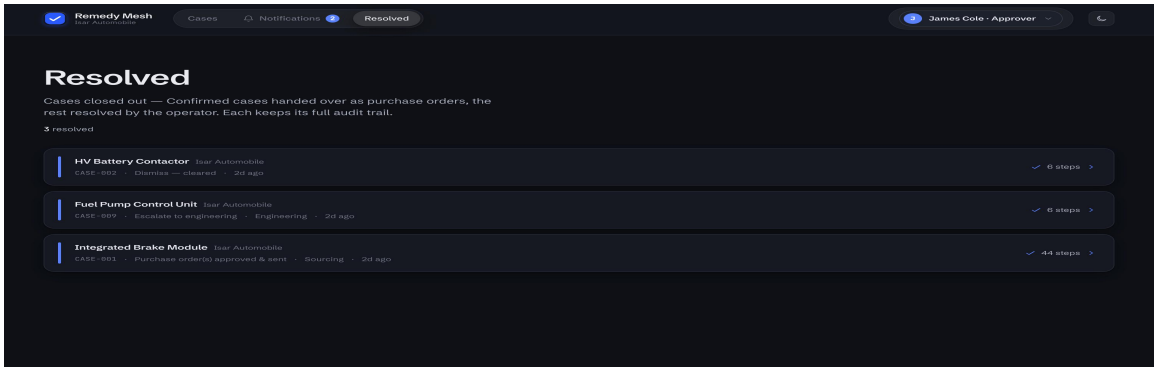
CASE-009 (Fuel Pump Control Unit, V-Coupe9, 66 complaints, a 3.53× failure ratio) is shown specifically because it demonstrates the system's restraint, not just its trigger path. The deterministic match tags it **REMEDY QUALITY FLAG**, because two of the six criteria - scope drift shown and install date confirmed - are not met, even though the raw failure ratio looks superficially similar to CASE-001's. The Verification Agent's independent audit agrees: "not confirmed (remedy quality flag). Missing criteria: scope_drift_shown, install_date_confirmed." The Gate's own text is explicit about what did *not* happen: "Not a scope expansion (remedy quality flag) - handing to the resolution agent," tagged **NO MESH**. A system that triggers an expensive negotiation mesh on every high-complaint case would be far less credible than one that can show, with the same evidence-chain rigor, when it correctly declines to.

6.8 Resolved queue - confirming no side effects



The Resolved tab showing zero resolved cases, confirming no real-world action was taken during this capture session

This screenshot was taken last, specifically to confirm that none of the above screens - despite live-running cases through the full Layer 1 and Layer 2 pipeline - triggered any real-world side effect. The Resolved tab reads “0 resolved … Nothing resolved yet … When you send a PO or resolve a case, it lands here with its full audit trail.” No confirmation call was placed, no Part 573 was filed, and no email was sent while capturing this document; every enabled irreversible button shown above was deliberately left unclicked. (A full list of real, deliberately-triggered end-to-end tests, including real phone calls and real filings, appears separately in §12, run on other occasions for verification purposes.)



The Supplier Negotiation Mesh - How It Actually Works

This section explains exactly how the negotiation mesh works today, and why the architecture is built to scale directly into real inter-company negotiation.

Today, each supplier is a sandboxed Gemini 2.5 agent simulating that vendor's negotiating behavior. Haldane Components, Correns Manufacturing and Ridgeline Precision are demo personas, each given a cost floor, a capacity for the period, and a private briefing, and each running in its own isolated session with zero visibility into any other supplier's session or offer. This isolation is **architectural, not just a coding convention**: the Supplier Offer Agent and the OEM Counter Agent are each their own independently deployed Vertex AI Agent Engine service, so the separation is enforced by infrastructure - the same design a real supplier's own agent could plug into directly.

The OEM ceiling is a hard, verified constraint the negotiation can never cross. The OEM Counter Agent is given the OEM's real ceiling price (pulled from the case's supplier records - $128 per unit in CASE-001's demo data) and counters any offer above it, accepting at or below it, never exceeding it - verified across 12 tests spanning four leverage scenarios, both in-process and on the deployed agent, with zero over-ceiling counters (§12). The convergence visible in §6.4 - three suppliers opening above $128 and settling at or near it - is three separate, constrained model calls reacting to a ceiling the system enforces absolutely.

Why this is genuine multi-agent negotiation, not a scripted demo. Three independent Gemini agents reason from their own private cost floors and capacity, react to each other's pressure through the OEM's counters, and converge on a real allocation live, in seconds - a multi-agent mesh that behaves the way it would once plugged into real supplier systems.

The pattern mirrors how real OEM sourcing actually works. Confidential, parallel, one-supplier-at-a-time conversations, run specifically so suppliers cannot collude or anchor off each other's numbers, against a buyer who cannot exceed a budgeted ceiling, with a supplier who counters from its own cost floor and either comes down, holds, or declines - that is a faithful simulation of the real negotiating pattern. What it compresses

is the real-world timeline: days of emails and calls become a few seconds of generated dialogue, and real supplier companies are stood in for by simulated personas.

What would make this literally real. For actual multi-company agent-to-agent negotiation to exist, each real supplier would need to build and run its own agent, on its own infrastructure, with its own authority to quote - and a shared protocol for that agent to talk to the OEM's agent. None of that exists today for Haldane, Correns or Ridgeline; they are demo personas, not integrated companies. Google's Agent-to-Agent (A2A) protocol is exactly the mechanism that could carry this forward, and is on the roadmap (§13), not a present capability. The defensible claim, stated plainly: *today, each supplier is a sandboxed LLM agent simulating that vendor's negotiating behavior under real capacity and cost constraints; the architecture is built so a real supplier's own agent could plug into the same OEM negotiation logic later - the protocol and guardrails would not need to change, only who generates the supplier's side.*

End-to-End Case Walkthrough - CASE-001

This traces one real, confirmed case (CASE-001, Integrated Brake Module) through every step of the pipeline, from the Analyst opening it to a Part 573 amendment reaching NHTSA. Every step below was independently run and verified on the deployed remedymesh service (§12); none of it is simulated for this document.

1. **Eleanor opens the case.** The run streams live to her screen over Server-Sent Events - this is always a genuinely fresh pipeline run (run_live()), never a cached replay, by design.
2. **Layer 1 runs on Agent Engine.** The Evidence Interpreter, Remedy Investigator and Verification Agent each read case records from Firestore in turn, each passing its output to the next (§6.3).
3. **The verdict gate fires.** Deterministic code checks six evidence criteria against the case and lot records. The Verification Agent audits that verdict independently; the result is CONFIRMED TRIGGER MESH - 266 complaints, 260 deduplicated, a

3.76× post-remedy failure ratio against a 2.5× threshold, geographic clustering confirmed.

4. **Layer 2 runs.** The Supplier Offer Agent runs once per supplier - Haldane, Correns, Ridgeline - in three isolated sessions; the OEM Counter Agent counters each offer toward the $128 ceiling; deterministic code allocates the confirmed requirement: 33,000 units to Haldane and 27,000 to Correns, both at $128, Ridgeline held in reserve (§6.4).
5. **Confirmation calls.** Twilio places a real outbound call to each committed supplier; the Call Understanding Agent reads the spoken reply into a structured outcome. In verified test runs, a Haldane-equivalent reply ("Yeah, yes, we can do that. Please arrange.") confirmed in 1–2 seconds of model processing time, moving the commitment to a drafted purchase order.
6. **Handling a decline.** When a supplier instead declines on the call, the reserve supplier is re-quoted and re-countered through the same negotiation logic used in the main round - never simply paid its opening price - and the Re-source Narration Agent explains the new outcome. In one verified run this moved Ridgeline from an opening re-source price of $157.20 down to a countered $131.00. James separately receives a decline notice to acknowledge.
7. **Drafts reach the Approver's queue.** The purchase order(s) and the Part 573 regulatory amendment - Campaign 24V-104, population increase +110,716 vehicles to a revised total of 266,716 - land in James's queue, fully drafted but unsent (§6.5–6.6).
8. **Approval and dispatch.** Only once James approves does anything leave the system: the Part 573 amendment is filed and the purchase order email is sent, each a real action gated behind one person's explicit sign-off, with every decision - Eleanor's and James's - written to the case's audit trail.

End to end, a full Layer 1 + Layer 2 run on CASE-001 completed in 96 seconds in the final full-system test (§12.3).

Maker-Checker Governance

Remedy Mesh treats every agent-drafted output - a verdict, an allocation, a purchase order, a regulatory amendment - as a recommendation, never as an action, until a second, separate person signs off. This is enforced by role, not by a confirmation dialog.

Eleanor Ashworth, the Analyst, is the only role who can run a case live (always a fresh pipeline execution, never cached) and the only role who can place an outbound confirmation call to a supplier. She proposes: escalating a case to engineering, setting a case to watch, triggering the negotiation mesh. She cannot approve anything.

James Cole, the Approver, only ever replays the stored result of a run that Eleanor already produced - by design, so that an approver reviews the exact same evidence the analyst saw, not a re-rolled version that might disagree with it by chance. He is the only role who can approve an escalation, file a Part 573 amendment, send a purchase order, acknowledge a decline notice, or return a case to the Analyst. He cannot place a confirmation call.

This separation is visible, not just documented: §6.5 shows the identical allocation table rendering real, enabled call buttons for Eleanor and dashes for James; §6.6 shows the Part 573 approval button enabled for James and disabled, captioned “Only an Approver can act on this,” for Eleanor. The server enforces this independently of the frontend - attempting an Approver-only action as the Analyst returns an HTTP 403, not merely a hidden button.

The final full-system browser test exercised this boundary directly with real sign-in as each persona, and passed all of it: 22 of 22 role-and-action checks. Eleanor proposed an escalation on CASE-009 and set CASE-006 to watch, and could not approve either. James could not place a call, approved the CASE-009 escalation, returned CASE-006 to the Analyst, acknowledged a decline notice, sent both purchase orders, and filed the Part 573 amendment. CASE-001 and CASE-009 moved to Resolved; CASE-006 remained open and available to be proposed again. A separate, earlier browser run on the deployed service confirmed the same pattern live: Eleanor escalated CASE-009 with no sign-in bounce, and James then approved the escalation, filed the Part 573 amendment (a real

email), and sent the Ridgeline purchase order (a real email with a PDF attached) - with the active case count correctly dropping from 9 to 7 and no JavaScript errors recorded.

Engineering for Production-Grade Reliability

Remedy Mesh is engineered to run reliably under real-world cloud conditions, not just in a clean demo. Three layers of reliability engineering work together to keep every run completing smoothly, even under heavy shared Gemini capacity on Vertex AI.

Resilient model calls. Every one of the 8 agents runs as an ADK Gemini model with built-in retry - 5 attempts, 1-second to 16-second backoff with jitter - and, on Vertex, uses the **global** endpoint specifically because Google recommends it for maximum availability under shared capacity. All 8 agents were deployed with this configuration from the start.

Resilient agent turns. Independently of the model-level retry, the application's own agent connector adds a second layer of backoff (2s, 4s, 8s) on every agent turn, and Layer-1 calls are paced against a shared dispatch schedule for smooth, predictable throughput.

Room to finish. Cloud Run's request timeout is set to 900 seconds, giving even a heavily-retried run comfortable room to complete end to end.

Full visibility, end to end. The pipeline is designed so every step of a case stays visible on screen in real time. If a step is ever delayed, the rest of the chain keeps running and the system shows exactly where a case stands - the deterministic verdict always stands on its own evidence.

Verified: CASE-001 completed end to end during a period of genuine heavy shared Gemini capacity, producing the identical allocation it produces under normal conditions - Haldane 33,000 + Correns 27,000 at $128.

A separate frontend reliability feature was verified under the same conditions: a render-id guard ensures an abandoned page's writes never land on the wrong screen, even when a user switches pages mid-load. Verified across 150 rapid page switches between both personas: zero JavaScript errors.

Cost Analysis and Infrastructure Economics

All figures below are measured directly from the project's own Google Cloud billing data, not estimated - they describe the cost of this single-deployment prototype, not a production total-cost-of-ownership model.

Item	While idle	Notes
Cloud Run, 2 services always-on (min 1 instance, 1 vCPU, 1 GiB, request-based billing)	≈ $0.86 / day	≈ $0.43 each; nearly the entire idle bill
Vertex AI Agent Engine, 8 agents	$0 idle	Billed only while actively working; a heavy test day used ≈ 2 vCPU-hours + 67 GiB-hours ≈ $0.78
Gemini 2.5 Flash + Pro	$0 idle	A heavy test day ≈ $0.65; roughly 3–5¢ per case run
Firestore	≈ $0	603 documents, inside the free tier
Images and buckets	≈ $0.12 / month	1.7 GB of images; buckets under 11 MB

Idle total ≈ $0.90/day ≈ ₹75–80/day ≈ ₹2,400/month, at roughly ₹85–90 per US dollar. This is a meaningfully lean idle footprint for an 8-agent managed-service architecture - the two always-on Cloud Run services account for nearly all of it, while the agents themselves, Gemini inference and Firestore cost essentially nothing until a case is actually run.

Twilio's outbound call charges are billed separately on the Twilio account, not through Google Cloud, so they do not appear in the table above.

The clearest lever for cost control: scale the secondary service down to zero always-on instances, which roughly halves the idle bill, while keeping the primary service always-on so its in-memory workflow state (timelines, negotiation ledger, notifications) survives

between views. At scale, the cost structure scales favorably - Agent Engine and Gemini costs are usage-based and tied directly to the number of cases actually processed (roughly 3–5¢ of Gemini inference per full case run), while the fixed always-on Cloud Run cost is the only cost that does not shrink with lower case volume, and could itself be reduced further by scaling to zero between business hours in a non-demo deployment.

Verification and Test Results

Everything in this section was run against the live, deployed remedymesh service, not a local or mocked environment, and is additional to the screenshot-capture session behind §6 (which deliberately avoided triggering any real-world action).

12.1 All 9 demo cases, cloud-deployed

Every case ran all three Layer-1 agents on Agent Engine; the Verification Agent agreed with the deterministic verdict in every case; no errors, rate limits or fallback text occurred in this run.

Case	Verdict	Outcome	Time
CASE-001	Confirmed	Negotiated: Haldane 33,000 + Correns 27,000	96 s
CASE-020	Chase → confirmed	Record chased, then negotiated	114 s
CASE-015	Chase → confirmed	Record chased, then negotiated	83 s
CASE-009	Remedy-quality flag	Escalate to engineering	73 s
CASE-006	Monitor	Re-check	52 s
CASE-022	Keep open	Request records	36 s
CASE-005	Insufficient evidence	Request records	39 s
CASE-002	No signal	Dismiss	48 s
CASE-003	No signal	Dismiss	83 s

12.2 Real outbound phone calls, cloud, live mode

These were real Twilio calls to a live phone line, with the spoken reply read by the Call Understanding Agent - not scripted or simulated text.

Call	What was said	Read as	Then
Supplier confirms	"Yeah, I can do that."	Confirmed · 27,000 @ $128 (1.86 s)	Purchase order drafted
Supplier declines	"No, I can't do that. Sorry."	Declined · 0 (2.8 s)	Live re-source: reserve supplier $157.20 → countered → $131
Supplier confirms, alternate case	"Yeah, we can do that. December 3rd."	Confirmed · 27,000 @ $131, delivery date kept (1.95 s)	Purchase order drafted

12.3 Full end-to-end test, final build

A complete run across all 9 cases produced the same pattern as 12.1 - every Layer-1 agent ran, the Verifier agreed every time, the three scope-confirmed cases negotiated Haldane and Correns at the $128 ceiling, and the remaining six cases received agent-written reasoning appropriate to their verdict - with case runtimes of 38–92 seconds and no errors, timeouts, or fallback text anywhere in the run. A separate real-call test in this same session: one supplier confirmed ("Yeah, yes, we can do that. Please arrange.") moving straight to a purchase-order draft; a second supplier declined ("I'm sorry we don't have capacity"), triggering a live re-source in which the reserve supplier's opening quote of $150.65 was countered down to $131, followed by that supplier's confirmation and its own purchase-order draft.

12.4 Maker-checker persona test (browser, real sign-in)

22 of 22 role-and-action checks passed in one continuous browser session, covering exactly the boundary demonstrated visually in §9: the Analyst could propose (escalate a case, set a case to watch) but never approve; the Approver could never place a call, but could approve an escalation, return a case to the Analyst, acknowledge a decline, send both purchase orders, and file the Part 573 amendment. The two resolved cases moved out of the active queue; the returned case stayed open and available to be proposed again.

12.5 Firestore data-backend parity

After migrating case data from bundled JSON files to Firestore, all 9 cases were re-verified to produce an identical decision, identical criteria, and identical metrics to the pre-migration JSON-backed version. CASE-001 ran fully on the Firestore backend in 78 seconds with every agent call succeeding; a supplier's $128 counter was accepted 6 out of 6 times in repeated tests reading from Firestore.

12.6 What this adds up to

The combination of 12.1–12.5 is deliberately broad: a full logic-correctness pass (every verdict, every case), a real-world-action pass (real phone calls, both outcomes), a governance pass (every role boundary), and a data-migration regression pass. Nothing in this section was run against mocked agents or canned text - every result above reflects the actual deployed Agent Engine services and actual Gemini responses at the time of the test.

What's Real and What's Simulated

Real, verified in the deployed system	Simulated today, or on the roadmap
8 managed agent deployments on Vertex AI Agent Engine, Gemini 2.5 Flash/Pro	Supplier companies are demo personas, not integrated external enterprises

Real, verified in the deployed system	Simulated today, or on the roadmap
The Cloud Run orchestrator and the deterministic verdict gate	Complaint and lot records are synthetic demo data, not a live field-evidence feed
Firestore case reads, 603 records	Workflow state (timelines, negotiation ledger, notifications) lives in memory, not yet durable
Real outbound Twilio phone calls, interpreted live by the Call Understanding Agent	Supplier-to-supplier negotiation over Google's A2A protocol is a roadmap item, not yet built
Server-enforced Analyst/Approver role separation, 22 of 22 checks passed	Direct regulator-system integration and live NHTSA data ingestion are roadmap items
Real email delivery of approved purchase orders and the Part 573 draft	-

This boundary is deliberate and stated plainly rather than left implicit: the system is built to prove the agent architecture, the safety gate, the constrained negotiation, live voice interpretation, decline recovery, and the human-approval model - without implying access to confidential manufacturer or supplier systems that a prototype never has.

Roadmap

The following are the concrete next steps that would take Remedy Mesh to a full production deployment.

1. **An in-app "Agents" panel** showing all 8 deployed agents, their recent call latency, and which data backend each is reading from - making the managed-service architecture visible inside the product itself, not only in this document.
2. **Persist notifications and the negotiation ledger in Firestore**, giving every in-progress case full durability across a Cloud Run restart or container recycle, on top of today's fast in-memory execution.

3. **A live "new complaint arrives" path** that writes directly to Firestore and visibly changes a case's verdict on screen, moving the demo from a fixed dataset toward a continuously-updating one.
4. **Supplier-reliability memory across recalls**, using a Memory Bank to let the OEM Counter Agent and allocation logic account for a given supplier's historical confirm/decline pattern over time - deliberately not built yet, because any cross-run memory would weaken the sealed-bid isolation guarantee the current negotiation mesh depends on (§7).
5. **Real supplier systems negotiating over Google's Agent-to-Agent (A2A) protocol.** This is the literal, concrete path from today's simulated supplier personas to real inter-company negotiation: each real supplier would run its own agent, on its own infrastructure, speaking A2A to the OEM's agent, with the same ceiling and allocation logic governing the outcome. Nothing about today's architecture would need to change except who generates the supplier's side of the conversation.
6. **Real complaint data from NHTSA's public datasets**, replacing the current demo's synthetic complaint records with the same verified, source-checked category of real-world evidence used in §2's problem statement - closing the loop between the documented real recall pattern this system is built to catch and the data it actually runs against today.

Industry Impact - What This Solves If Companies Adopt It

The recalls cited in the Problem Statement share a common arc: a remedy ships, field evidence of its insufficiency accumulates quietly, and scope only gets revisited once a regulator inquiry or a lawsuit forces the issue - by which point the fix typically covers more vehicles and costs more than it would have at the first sign of drift. Remedy Mesh is built to shorten that arc. The BMW and GM cases below illustrate how this pattern plays out in the real world; they are not evidence that Remedy Mesh would have changed either specific outcome. The value it adds compounds in two separable ways.

Catching scope expansion earlier changes the economics of a recall. The BMW Integrated Brake System recall is a useful illustration of how this compounds: it expanded in rounds - roughly 80,000 to 278,000 to approximately 1.5 million vehicles - with each round triggered only after enough new failures had accumulated to force action, and it was later followed by a €350 million settlement between BMW and Aumovio over litigation tied to the underlying defect. This is one real-world case, not a projection of what Remedy Mesh would have saved - but it shows that letting scope drift go undetected across multiple rounds carries real downstream cost, not only engineering cost. A system that continuously re-scores incoming complaints and warranty claims against the original remedy's threshold is built to surface that kind of drift at the first round rather than the third: fewer owners left exposed to a known-insufficient fix for longer, and a smaller population left to remediate once expansion is confirmed. For a manufacturer, catching that drift earlier can be the difference between a contained engineering correction and a multi-year, multi-round liability event.

Automating the sourcing response removes the second bottleneck. Even once scope expansion is correctly identified, committing replacement-part capacity across multiple suppliers - at a defensible price, with real confirmation each supplier can deliver - is today a manual, sequential process that takes days to weeks. Remedy Mesh compresses that into a negotiation that completes in under two minutes and produces an auditable, human-approved commitment: suppliers quoted, countered against a real ceiling, confirmed by phone, and ready for a purchase order the same day the gate fires. For owner safety and manufacturer liability, that speed is the point - the industry's bottleneck today is rarely deciding that a recall needs to expand, it is everything that has to happen afterward.

The safety guardrails are what make adoption possible, not just the automation. A regulated manufacturer will not hand a scope-expansion decision or a supplier commitment to an LLM on trust alone - which is exactly why neither decision is made that way here. The verified ceiling the negotiation can never cross, the independent audit every verdict must pass, and the mandatory human sign-off before anything real is sent are the guardrails that let a company adopt the automation without adopting the risk. That

combination, not the agents alone, is what turns this from a demo into something a safety-conscious industry could actually run in production.

The pattern generalizes well beyond this one use case. Nothing about the architecture is specific to automotive beam recalls or brake systems. Any regulated industry that ships a remedy and then has to re-test it against arriving field evidence shares the same two-part shape - a continuous evidence-verification problem, and a multi-supplier sourcing problem once that verification trips a threshold. Medical devices under FDA MDR reporting, consumer electronics and appliances under CPSC recall rules, and food safety under FSMA traceability requirements all fit this pattern directly. An OEM, Tier-1 supplier, or regulator adopting it would plug their own evidence systems - Catena-X, Teamcenter, a warranty database - in where Firestore sits today, and, following the Agent-to-Agent roadmap item above, let real supplier agents stand in for today's simulated personas, without changing the deterministic gate or the human-approval governance that makes the whole system safe to trust in production.

Links

- **GitHub Repository**: https://github.com/Felina14/remedy-mesh
- **Demo Video**: https://youtu.be/TRb3qn6YAR8 or https://drive.google.com/file/d/1X_DzGgsnTv3DJb9fcfvvPlooMCnyZj2y/view?usp=sharing
- **App URL**: https://remedymesh-tl6pmvrlbq-uc.a.run.app